Soluciones: Punteros

Programación I, Semana 11

Ejercicios obligatorios

Ejercicio 1: Tabla de memoria

Línea	`x`	`y`	`p`	Qué pasó
1	5	-	-	Se crea `x`.
2	5	8	-	Se crea `y`.
3	5	8	1000	`p` guarda la dirección de `x`.
4	7	8	1000	`*p` es `x`: cambia `x`.
5	7	8	1004	`p` ahora apunta a `y`.
6	7	15	1004	`*p` es `y`: 8 + 7 = 15.

Al final, `*p` es `y`: imprime **15**.

Errores frecuentes:

- Escribir 7 en la columna `p` en la línea 4: `*p = 7` no cambia `p`, cambia la variable a la que apunta.
- En la línea 6, cambiar `x`: desde la línea 5, `p` apunta a `y`.

Ejercicio 2: ¿Qué imprime?

a) Imprime `10 10`. `q = p` copia la **dirección**: `p` y `q` apuntan los dos a `a`. Cambiar `*q` cambia `a`, y `*p` también lo ve.

b) Imprime `9 8`. `p = v` hace que `p` apunte al primer elemento de `v`. `p[1]` es el mismo elemento que `v[1]`, así que `v[1]` pasa a 9. `p[3]` es `v[3]`, que vale 8.

c) Imprime `4 8`. `x` es una copia de `a`: `a` no cambia. `y` recibe la dirección de `b`: `*y = *y * 2` cambia `b` a 8.

Ejercicio 3: Intercambiar

```
void intercambiar(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 3;
    int y = 9;
    intercambiar(&x, &y);
    std::cout << x << " " << y;    // imprime 9 3
    return 0;
}
```

Por qué: la función recibe las direcciones de `x` y de `y`, así que con `*a` y `*b` modifica las variables de `main`. `temp` guarda el valor de `x` antes de perderlo.

Errores frecuentes:

- `int temp = a;` o `a = b;` sin *: copia direcciones, no valores. Las variables de `main` no cambian.
- `intercambiar(x, y)`: la función espera direcciones; hay que escribir `&x` y `&y`.

Ejercicio 4: Encontrar los errores

N.º	Error	Corrección
1	`p.precio`: `p` es un puntero, no un struct.	`p->precio = p->precio - p->precio * porcentaje / 100;`
2	`aplicarDescuento(lapiz, 50)`: la función espera una dirección.	`aplicarDescuento(&lapiz, 50)`
3	`int* c = cantidad;`: a un puntero se le asigna una dirección, no un `int`.	`int* c = &cantidad;`
4	`int* total;` no apunta a ninguna variable: `*total = ...` escribe en una dirección desconocida.	Usar una variable: `int total = *c * 2;` y luego imprimir `total`.

Ya corregido, el lápiz pasa de 2 a 1, `total` es $3 \times 2 = 6$, y el programa imprime **1 6**.

Sobre el error 4: también es correcto declarar `int t;` y luego `int* total = &t;`. Lo importante es que un puntero siempre apunte a una variable que existe (o sea `nullptr`) antes de usar *.

Ejercicios intermedios

Ejercicio 5: Cuenta bancaria

```
void depositar(Cuenta* c, double monto) {
    c->saldo = c->saldo + monto;
}

bool retirar(Cuenta* c, double monto) {
    if (monto > c->saldo) {
        return false;
    }
    c->saldo = c->saldo - monto;
    return true;
}
```

Saldo inicial 100. `depositar(&cuenta, 50)`: 150. `retirar(&cuenta, 200)`: 200 es mayor que 150, devuelve **false** y no cambia nada. `retirar(&cuenta, 30)`: devuelve **true** y deja **120**.

Errores frecuentes:

- `c.saldo` en lugar de `c->saldo`.
- Restar el monto antes de revisar si alcanza: el saldo quedaría negativo.
- `if (monto >= c->saldo)`: con ese `>=` no se podría retirar todo el saldo.

Reto

Ejercicio 6: Buscar una cuenta

a)

```
Cuenta* buscarCuenta(Cuenta c[], int n, std::string titular) {
    Cuenta* encontrada = nullptr;
    for (int i = 0; i < n && encontrada == nullptr; i++) {
        if (c[i].titular == titular) {
            encontrada = &c[i];
        }
    }
    return encontrada;
}
```

Es la búsqueda de siempre, pero en lugar de `pos = -1` se usa `nullptr`, y en lugar de guardar `i` se guarda la dirección `&c[i]`.

b)

```
Cuenta* m = buscarCuenta(banco, N, "Maria");
if (m != nullptr) {
    depositar(m, 20);       // m ya es una dirección: sin &
}
```

El saldo de `banco[2]` pasa de 80 a **100**: el puntero apunta al elemento del arreglo, no a una copia.

Errores frecuentes:

- `depositar(&m, 20)`: `m` ya es un puntero. `&m` sería la dirección del puntero, que no es una `Cuenta*`.
- Usar `m->saldo` sin revisar antes si `m` es `nullptr`.